



ECOLE SUPERIEURE DE GESTION D'INFORMATIQUE ET DES SCIENCES

PLATEFORME ANFA
RAPPORT TECHNIQUE DE REFERENCE

Cours	Cloud & Big Data
Chargé de cours	M. AKPAGNONITE Denis
Auteur	GBOSSOU Folly Sitou Carlo, (GitHub: davilla1993)
Année académique	2025 - 2026
Date de remise	05 Août 2026

Table des matières

Introduction	3
Partie 1 — Cartographie de ma plateforme	3
1.1 Le chemin de la donnée.....	3
1.2 Preuves par brique	4
Partie 2 — Journal des arbitrages.....	8
2.1 Kind plutôt que Minikube (séance 3)	8
2.2 Provider Docker de Terraform plutôt qu'un provider cloud (séance 4).....	9
2.3 Image Airflow personnalisée plutôt que le socket Docker prescrit (séance 6)	9
2.4 Logique métier isolée dans anfa_logic.py plutôt que testée directement dans Airflow (séance 8).....	10
2.5 Fichier sentinelle /tmp/anfa_en_panne plutôt qu'un docker stop (séance 9).....	10
Partie 3 — Analyse du fil rouge « ça tourne mais c'est inutile ».....	10
3.1 Panne 1 — Airflow : un DAG qui « réussit » en propageant une donnée vide (séance 6)10	
3.2 Panne 2 — Monitoring : fraîcheur des données figée sans que rien ne « plante » (séance 9).....	11
3.1 Panne 3 — MLOps : un modèle qui répond de plus en plus mal, sans jamais planter (séance 10).....	11
3.2 Le fil transversal : garder la trace plutôt que présumer	12
Partie 4 — Le passage au cloud réel.....	12
4.1 MinIO → Amazon S3	12
4.2 Kind → EKS / GKE.....	13
4.3 Spark standalone → EMR / Dataproc.....	13
4.4 Synthèse.....	14
Partie 5 — Souveraineté et conformité.....	14
5.1 Le scénario	14
5.2 Où héberger ces données ?	14
5.3 Ce qui manque encore.....	15
Conclusion.....	15

Introduction

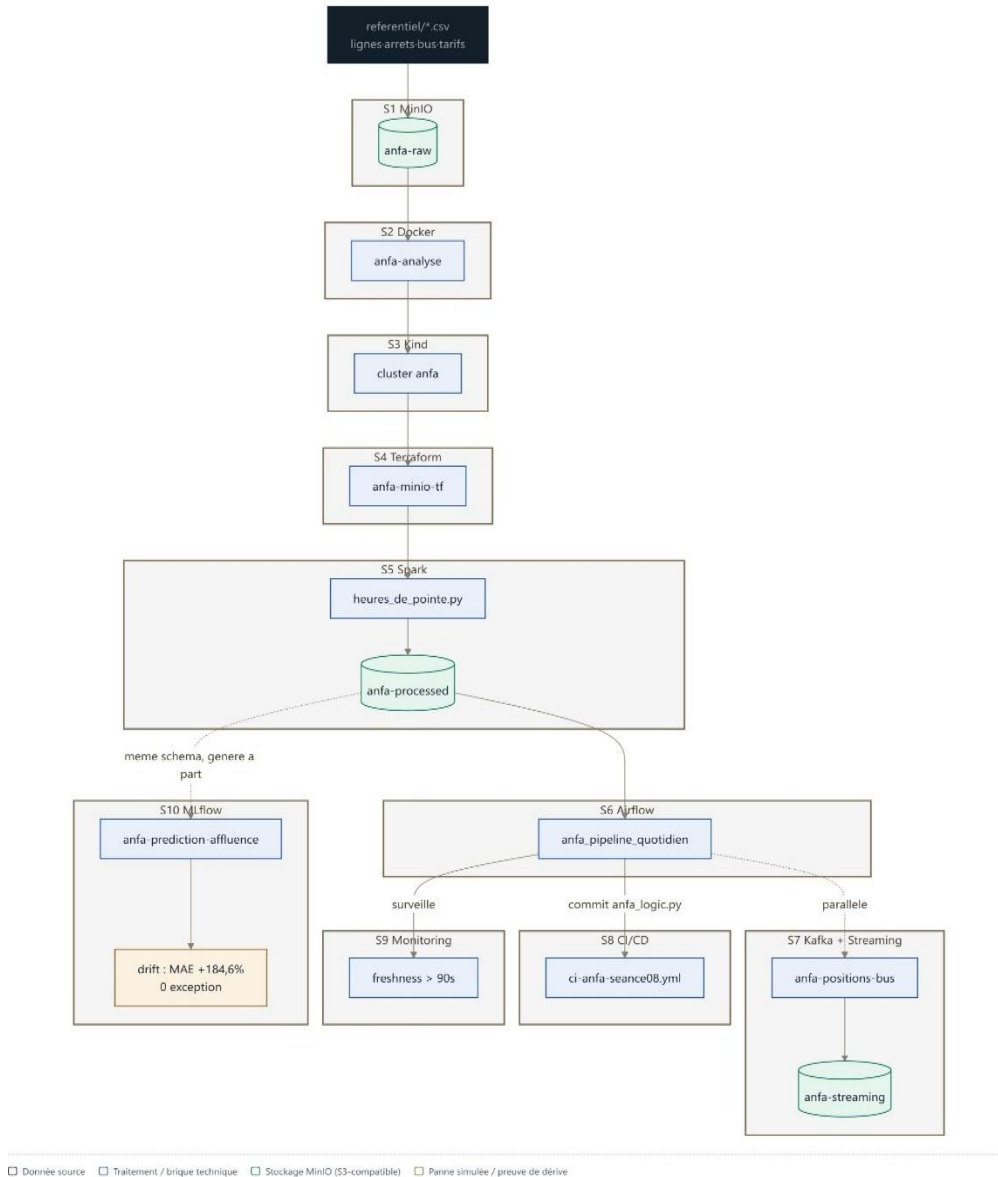
Anfa est une société fictive de transport public urbain à Lomé qui exploite environ une centaine de bus sur une douzaine de lignes. Ce semestre, on m’a demandé de construire, brique par brique, la plateforme data qui doit lui permettre de passer d’une exploitation à l’aveugle à un pilotage par la donnée : savoir où sont les heures de pointe, prévoir l’affluence, réagir en temps réel, et le faire sans jamais perdre de vue que ce qu’on construit doit rester légal, gouverné et défendable devant un conseil d’administration.

Ce rapport n’est pas un résumé de cours. C’est mon bilan d’ingénieur sur *ma* plateforme — celle que j’ai réellement déployée, cassée, réparée et documentée séance après séance, avec mes propres noms de ressources, mes propres captures d’écran et mes propres pannes. Il répond aux cinq parties demandées : la cartographie de ce que j’ai construit, le journal des choix techniques que j’ai dû trancher, une analyse des pannes silencieuses que j’ai moi-même provoquées, un test de la promesse « transposable tel quel vers le cloud », et enfin un jugement argumenté sur l’hébergement et la conformité des données sensibles d’Anfa.

Partie 1 — Cartographie de ma plateforme

1.1 Le chemin de la donnée

Tout part de quatre fichiers CSV : lignes.csv, arrêts.csv, bus.csv, tarifs.csv — le référentiel de base d’Anfa (11 lignes, 92 arrêts, 99 bus, 11 tarifs). Depuis ce socle, la donnée traverse dix briques que j’ai assemblées une à une :



Ce chemin illustre une chose que je n'avais pas anticipée en séance 1 : chaque brique ne remplace pas la précédente, elle s'appuie dessus. Le MinIO de la séance 1 sert encore de socle en séance 10. Le heures_de_pointe.py écrit en séance 5 est réutilisé tel quel par le DAG Airflow de la séance 6. C'est une vraie plateforme cumulative, pas dix TP indépendants.

1.2 Preuves par brique

Une capture par brique, prise dans mon propre environnement, avec mes propres noms de ressources (fichiers sources dans rapport-final/captures/). Chaque figure est insérée à la suite de sa légende.

Figure 1 — MinIO (S1). Bucket anfa-raw, préfixe référentiel/, 4 CSV uploadés.

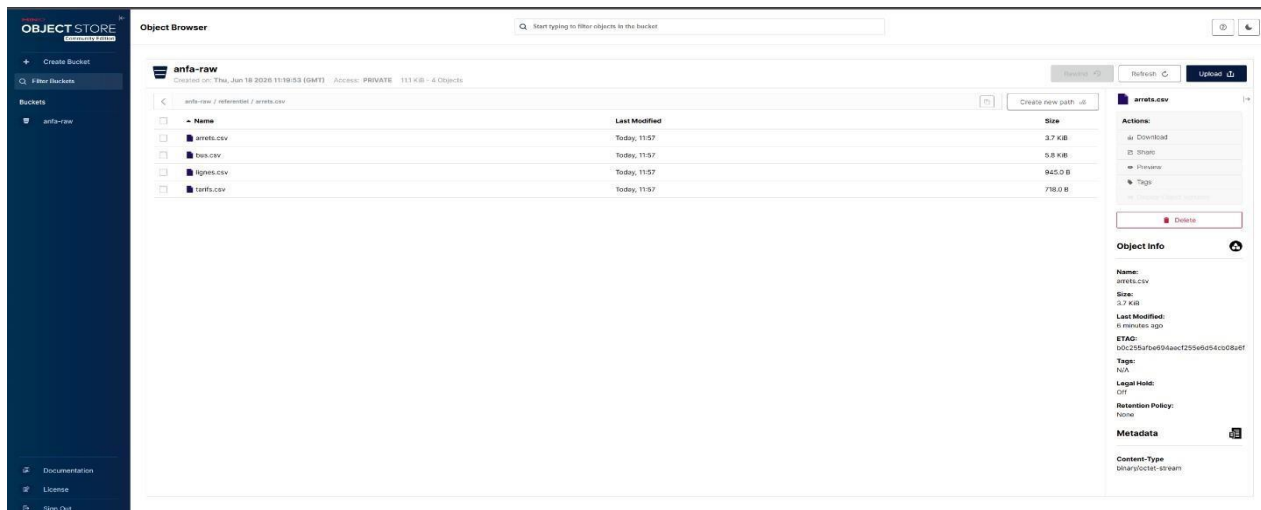


Figure 2 — Docker (S2). Stack à 3 services (MinIO + Jupyter + anfa-app), tous healthy.

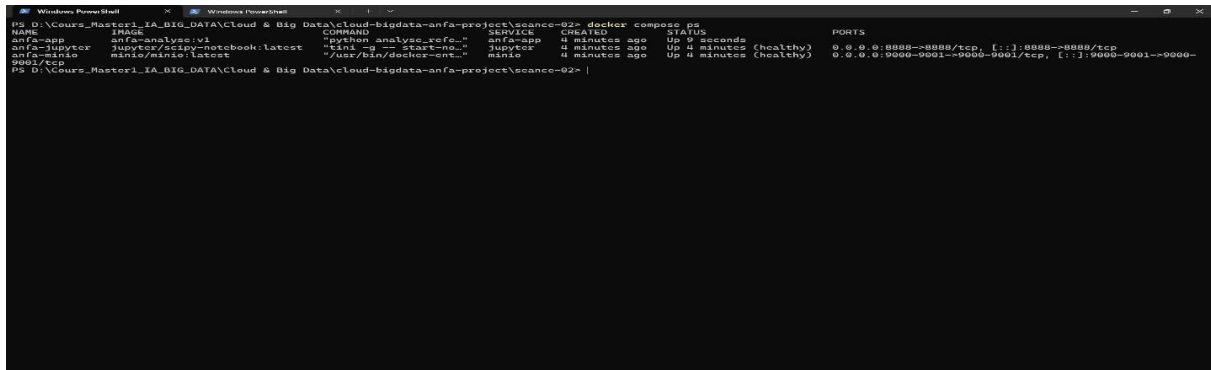


Figure 3 — Kubernetes / Kind (S3). Deployment minio scalé à 3 replicas.

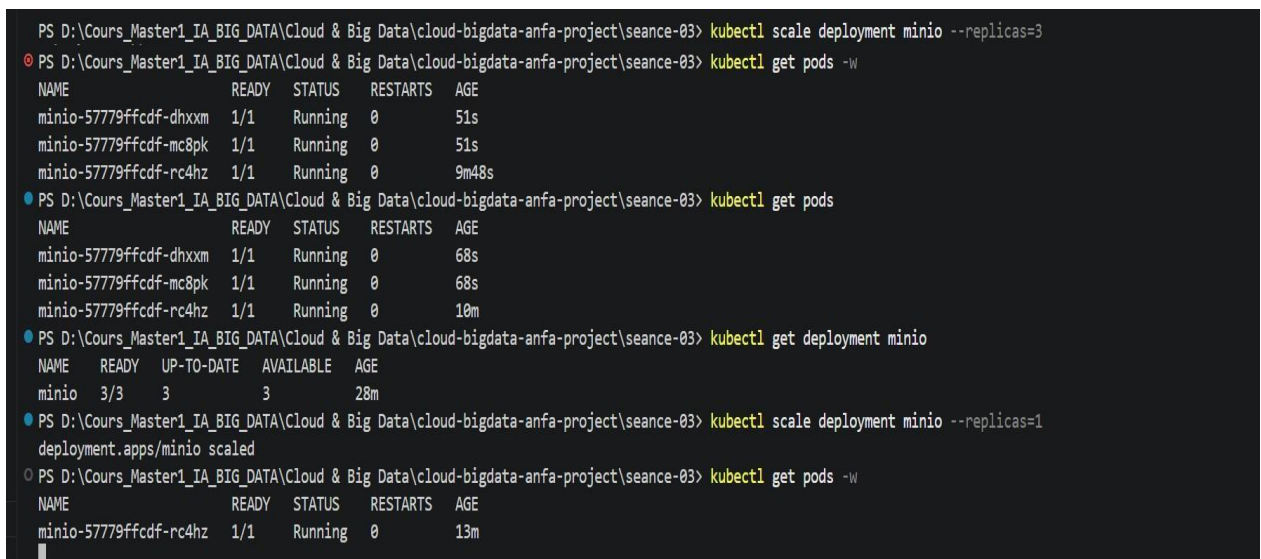


Figure 4 — Terraform (S4). Plan: 3 to add, 0 to change, 0 to destroy → Apply complete!

```
+ volumes {
+   container_path = "/data"
+   volume_name    = "anfa-minio-data-tf"
+   # (2 unchanged attributes hidden)
+ }
+ }

# docker_network.anfa_net will be created
+ resource "docker_network" "anfa_net" {
+   driver      = (known after apply)
+   id          = (known after apply)
+   internal    = (known after apply)
+   ipam_driver = "default"
+   name        = "anfa-network"
+   options     = (known after apply)
+   scope       = (known after apply)
+ }
+ ipam_config (known after apply)
+ }

# docker_volume.minio_data will be created
+ resource "docker_volume" "minio_data" {
+   driver      = (known after apply)
+   id          = (known after apply)
+   mountpoint  = (known after apply)
+   name        = "anfa-minio-data-tf"
+ }
+ }

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_volume.minio_data: Creating...
docker_network.anfa_net: Creating...
docker_volume.minio_data: Creation complete after 0s [id=anfa-minio-data-tf]
docker_network.anfa_net: Creation complete after 2s [id=7c1baabdcbbb38b8d5096f918668d37b15f094b16d63535968bf2fe040ca39f]
docker_container.minio: Creating...
docker_container.minio: Creation complete after 0s [id=4adf0e98ee96e7591ae0318350f7ccbb863bc7556194bcd44334e355c54de9b]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS D:\Cours_Master1_IA_BIG_DATA\Cloud & Big Data\cloud-bigdata-anfa-project\seance-04> |
```

Figure 5 — Spark (S5). Cluster standalone, 2 workers actifs sur le dashboard spark-master.

Spark Master at spark://1007e44e3013:7077

URL: spark://1007e44e3013:7077
 Active Workers: 2
 Cores in use: 2 / Total: 2 / Used
 Memory in use: 2.0 GB / Total: 8.0 GB / Used
 Resources in use: 1 / Total: 1 / Used
 Applications: 0 / Running: 0 / Completed: 0
 Drivers: 0 / Running: 0 / Completed: 0
 Status: Active

Worker ID	Address	State	Cores	Memory	Resources
worker-20200404141004-171223-0-00000	172.23.0.100041	Active	1 (0 Used)	1024.0 MB (0.0 / 8.0 Used)	
worker-20200404141004-171223-0-00000	172.23.0.100041	Active	1 (0 Used)	1024.0 MB (0.0 / 8.0 Used)	

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
No running applications.								

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
No completed applications.								

The screenshot shows the Apache Kafka UI for a cluster named 'anfa-cluster/brokers'. The 'Brokers' tab is selected, displaying a table of brokers. The table has the following columns: Name, Address, Controller, Version, Online, Uptime, In Sync Replicas, and Out of Sync Replicas. There are three brokers listed: kafka-0, kafka-1, and kafka-2. All three brokers are online and have 1 in-sync replica and 0 out-of-sync replicas.

Name	Address	Controller	Version	Online	Uptime	In Sync Replicas	Out of Sync Replicas
kafka-0	10.0.0.1	1	3.8.0	Online	1h 1m	1	0
kafka-1	10.0.0.2	1	3.8.0	Online	1h 1m	1	0
kafka-2	10.0.0.3	1	3.8.0	Online	1h 1m	1	0

Triggered via push 2 minutes ago

davilla1993 pushed

o- e4e3bc4

seance-08

Status

Success

Total duration

21s

Artifacts

-

ci-anfa-seance08.yml

on: push

✔ Lint et tests unitaires

10s

✔ Deployer le DAG valide (sim...

5s

Annotations

2 warnings

⚠ Lint et tests unitaires

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.com/actions/runner/issues/2024>

Show more

Figure 9 — Monitoring (S9). 4 cibles Prometheus à l'état UP (modèle *pull*).

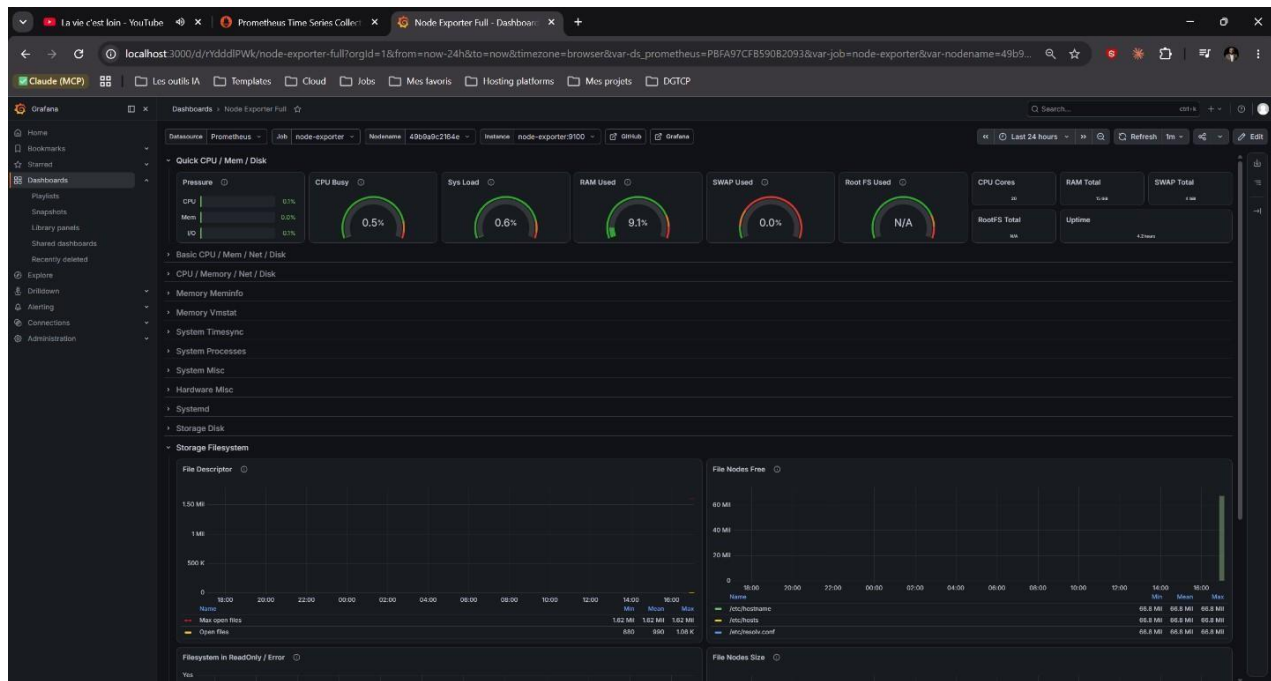
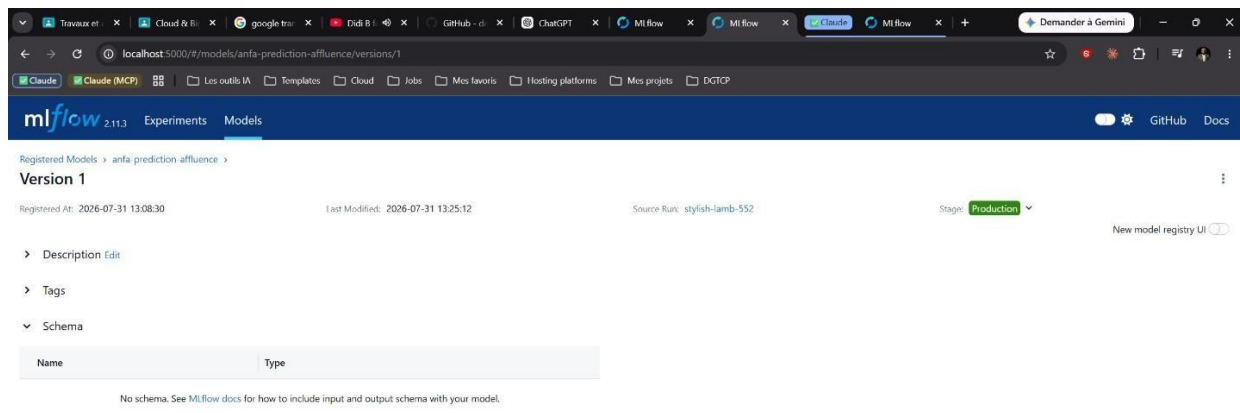


Figure 10 — MLOps (S10). Modèle anfa-prediction-affluence en statut Production dans le Registry.



Partie 2 — Journal des arbitrages

Cinq choix techniques réels rencontrés pendant le semestre, avec l'alternative écartée et ce qui casserait si j'avais fait l'inverse.

2.1 Kind plutôt que Minikube (séance 3)

Décision. Déployer MinIO sur un cluster Kubernetes local avec **Kind** (Kubernetes IN Docker), où chaque nœud est lui-même un conteneur Docker.

Alternative écartée. Minikube, qui fait tourner Kubernetes dans une VM dédiée (ou dans un conteneur selon le driver).

Ce qui casserait à l'inverse. Avec Minikube, l'accès réseau depuis l'hôte est parfois plus direct selon le driver choisi, mais le modèle mental « les nœuds K8s sont des conteneurs Docker » aurait été moins évident. Kind reste aussi plus léger à instancier/détruire en boucle pendant un TP, ce qui compte quand on refait la manipulation plusieurs fois pour un rendu.

2.2 Provider Docker de Terraform plutôt qu'un provider cloud (séance 4)

Décision. Décrire l'infrastructure MinIO en HCL avec le provider communautaire kreuzwerker/docker, qui pilote un démon Docker local plutôt qu'une API cloud.

Alternative écartée. Un provider cloud (AWS, GCP, Azure, OVHcloud).

Ce qui casserait à l'inverse. Un provider cloud m'aurait obligé à créer un compte, une carte bancaire, et à gérer des identifiants API dès la séance 4 — alors que l'objectif pédagogique de cette séance est le *workflow* Terraform (init/plan/apply/destroy, state, variables), pas la relation avec un cloud précis. Le mécanisme reste rigoureusement identique : mon main.tf se transposerait ressource par ressource vers un équivalent cloud. Ce que j'aurais perdu avec un provider cloud local uniquement : rien côté apprentissage Terraform, mais un vrai risque financier si j'avais mal géré le terraform destroy en fin de séance — avec le provider Docker, une ressource oubliée coûte du disque local, pas une facture.

2.3 Image Airflow personnalisée plutôt que le socket Docker prescrit (séance 6)

Décision. Construire une image Airflow sur mesure (Dockerfile.airflow, JRE headless + pyspark==3.5.8 embarqués) et lancer spark-submit directement depuis le conteneur Airflow en mode client.

Alternative écartée. La méthode prescrite par le TP : piloter spark-submit depuis le conteneur *spark-master*, via le SDK Docker Python (docker.from_env() + container.exec_run(...)), en montant le socket /var/run/docker.sock dans Airflow.

Honnêteté sur ce choix. Je dois être transparent : ce n'était pas un choix architectural conscient au moment où je l'ai fait. Je cherchais à résoudre une erreur JAVA_HOME is not set et j'ai construit une image avec un JRE sans réaliser sur le moment que je m'écarterais de la méthode du TP. Ce n'est qu'en relisant mon propre travail pour ce rapport que j'ai vu l'écart.

Ce qui casserait à l'inverse (ou : ce que mon choix a coûté). La méthode du socket Docker évite de dupliquer les dépendances Spark côté Airflow ; mon approche introduit un couplage de version fort entre le pyspark installé dans l'image Airflow (driver) et la version du cluster Spark (apache/spark:3.5.8) — couplage qui a provoqué mon troisième bug (InvalidClassException, désalignement 3.5.0 vs 3.5.8). À l'inverse, monter le socket Docker dans Airflow expose le démon Docker de l'hôte au conteneur Airflow — un vecteur d'attaque plus large en cas de compromission d'un DAG. Mon image custom n'a pas ce problème. Les deux approches ont donc un vrai compromis : couplage de version contre surface d'attaque élargie. Je retiendrais la mienne en production, mais en épinglant la version pyspark de façon disciplinée dès le départ plutôt qu'après un incident.

2.4 Logique métier isolée dans anfa_logic.py plutôt que testée directement dans Airflow (séance 8)

Décision. Extraire les fonctions pures (`verifier_liste_fichiers`, `construire_message_notification`, etc.) dans un module Python `anfa_logic.py`, sans dépendance à Airflow, boto3 ou MinIO, pour pouvoir les tester avec pytest dans la CI sans installer apache-airflow.

Alternative écartée. Tester la logique directement à l'intérieur des PythonOperator du DAG, en important Airflow dans les tests (ou en ne testant rien du tout, comme c'était le cas avant la séance 8).

Ce qui casserait à l'inverse. Installer apache-airflow en CI juste pour exécuter des tests unitaires est lourd (plusieurs dizaines de dépendances, minutes de setup) et lent - à l'échelle du pipeline CI (mon job `valider-dag` tourne en 9 secondes en cas d'échec), ça changerait complètement le temps de feedback. Pire : sans séparation, un bug dans la logique métier (comme ma conversion Ko volontairement cassée en /1000 puis testée une seconde fois en /1005) serait mélangé avec des questions d'infrastructure Airflow (connexions, variables d'environnement) — la CI deviendrait incapable de dire *quoi* a cassé. C'est exactement le scénario de Mawuli du CM : un changement non testé qui atteint la production sans qu'on sache pourquoi.

2.5 Fichier sentinelle /tmp/anfa_en_panne plutôt qu'un docker stop (séance 9)

Décision. Simuler une panne silencieuse du pipeline Anfa en laissant l'exportateur de fraîcheur tourner, mais en gelant la mise à jour de sa métrique via un fichier sentinelle (`/tmp/anfa_en_panne`) plutôt qu'en arrêtant le conteneur. **Alternative écartée.** `docker stop anfa-`

`freshness-exporter`.

Ce qui casserait à l'inverse. Un `docker stop` ferait disparaître la cible du scraping Prometheus : la série `up{job="anfa-freshness"}` passerait à 0, ce qui est une panne *visible* et déjà couverte par le monitoring système classique. Ça ne démontrerait rien de nouveau. Le fichier sentinelle simule exactement le cas réel qui motive toute la séance 9 : le processus reste vivant, répond toujours sur `/metrics`, la cible reste UP dans Prometheus - mais l'horodatage `anfa_dernier_traitement_timestamp` ne bouge plus. C'est la différence entre « le système est éteint » (facile à détecter) et « le système tourne mais ne produit plus rien d'utile » (la vraie panne silencieuse). Sans ce choix, je n'aurais pas pu observer ce que la métrique de fraîcheur apporte réellement par rapport à un simple statut UP/DOWN.

-Partie 3 — Analyse du fil rouge « ça tourne mais c'est inutile »

Trois séances de ce cours (S6, S9, S10) répètent la même leçon sous trois formes différentes : un statut `success` vert peut mentir. Voici les trois pannes que j'ai moi-même provoquées et pourquoi chacune est passée inaperçue des indicateurs habituels.

3.1 Panne 1 — Airflow : un DAG qui « réussit » en propageant une donnée vide (séance 6)

J'ai injecté un bug volontaire dans la tâche `verifier_resultats` de mon DAG `anfa_pipeline_quotidien` (raise `ValueError` si le bucket `anfa-processed/heures_de_pointe/` est

vide). Résultat observé dans l'UI Airflow : la tâche `verifier_resultats` passe au rouge, et notifier passe à l'orange (`upstream_failed`) au lieu de s'exécuter aveuglément. C'est le comportement voulu - mais ce qui compte ici, c'est ce qui se serait passé **sans** cette vérification : sans la tâche `verifier_resultats`, un job Spark qui écrit un fichier Parquet vide (parce que le job en amont, `generer_trajets`, a échoué silencieusement ou produit un fichier tronqué) n'aurait déclenché *aucune* erreur. Le DAG entier serait passé au vert, alors que le dashboard des heures de pointe n'aurait plus rien reçu de neuf. C'est exactement le scénario que le CM de la séance 9 attribue à Awa : `kubectl get pods` tout Running, logs Airflow en success, mais un fichier de trajets vide en amont.

3.2 Panne 2 — Monitoring : fraîcheur des données figée sans que rien ne « plante » (séance 9)

J'ai simulé la panne silencieuse elle-même (voir Partie 2.5) : le processus `anfa-freshness-exporter` reste vivant, répond toujours 200 OK sur `/metrics`, la cible Prometheus reste à l'état UP — mais `time()` - `anfa_dernier_traitement_timestamp` grimpe sans jamais redescendre. Aucune métrique système (CPU, RAM, statut du conteneur) n'aurait permis de détecter ce cas. C'est la distinction centrale de cette séance : les métriques système surveillent la machine, une métrique métier comme la fraîcheur surveille si le travail attendu a réellement eu lieu.

Point de rigueur que j'assume dans ce rapport : la capture censée montrer l'alerte à l'état Firing dans mon rendu de séance 9 montre en réalité la page des règles filtrée sur `state:firing` avec le message « No matching rules found » — probablement prise après le retour à l'état Normal, l'alerte se résolvant en moins d'une minute selon le TP. Le code de simulation et le mécanisme d'alerte fonctionnent (seuil 90s, évaluation 10s, persistance 30s), mais la preuve visuelle précise de l'état Firing manque à mon dossier. Je préfère le signaler ici plutôt que de laisser une capture dont la légende ne correspond pas exactement à son contenu.

3.1 Panne 3 — MLOps : un modèle qui répond de plus en plus mal, sans jamais planter (séance 10)

Pour vérifier concrètement le problème de Kossi (CM séance 10 — son modèle de prédiction d'affluence se trompe de plus en plus depuis l'ouverture de 2 nouvelles lignes en avril, « rien ne plante, tout est vert »), j'ai reproduit ce scénario sur mon propre modèle `anfa-prediction-affluence`. J'ai chargé la version en statut Production depuis le Model Registry et je l'ai évalué sur deux jeux de données : un jeu « baseline » avec les 12 lignes connues à l'entraînement, et un jeu simulant l'ouverture de 2 nouvelles lignes (L13, L14) jamais vues par le modèle.

	MAE	R ²
Baseline (12 lignes connues)	2,18	0,972
Après ouverture de L13/L14	6,19	0,760

Le MAE se dégrade de **+184,6 %** — et surtout, **aucune exception n'est levée nulle part**. Le `OneHotEncoder(handle_unknown="ignore")` de mon pipeline scikit-learn encode silencieusement les lignes inconnues à zéro plutôt que de faire échouer la prédiction : le modèle continue de répondre, juste de moins en moins bien. J'ai loggé ce run de comparaison dans MLflow

(monitoring-drift-ouverture-L13-L14, 5 métriques) pour qu'il soit consultable comme n'importe quel autre run - capture 10-mlflow-drift-monitoring.png.

anfa-prediction-affluence >
monitoring-drift-ouverture-L13-L14

Overview Model metrics System metrics Artifacts

Duration	315ms
Datasets used	—
Tags	type:run: monitoring_drift scenario: ouverture_2_nouvelles_lignes_avril
Source	simuler_data_drift.py 566b208269c7e2cc1bd55992049332411874ef
Logged models	—
Registered models	—

Parameters (3)

Parameter	Value
modele_evalue	anfa-prediction-affluence/Production
lignes_baseline	L01,L02,L03,L04,L05,L06,L07,L08,L09,L10,L11,L12
lignes_nouvelles	L13,L14

Metrics (5)

Metric	Value
mae_baseline	2.1751094057567038
r2_baseline	0.9723680609400921
mae_apres_drift	6.190084484676406
r2_apres_drift	0.7602767376251087
degradation_mae_pct	184.58727033654307

3.2 Le fil transversal : garder la trace plutôt que présumer

Ces trois pannes partagent un mécanisme identique : un système qui *fonctionne* au sens technique (pas de crash, pas d'exception, pas de conteneur DOWN) peut être en train d'échouer à son objectif métier. Le seul moyen de le savoir est d'instrumenter la bonne métrique — pas d'attendre qu'une erreur se déclenche d'elle-même.

Ce même principe traverse deux autres briques que je n'ai pas présentées comme des « pannes » mais qui relèvent exactement de la même logique : les **offsets Kafka** (séance 7) sont la mémoire de ce qu'un consumer a déjà lu — sans eux, un redémarrage de consumer ne saurait pas où reprendre et présumerait soit avoir tout traité, soit devoir tout retraiter, sans certitude ; le **state Terraform** (séance 4) est la mémoire de ce que l'infrastructure réelle contient — sans lui, terraform plan devrait présumer l'état du monde au lieu de le vérifier contre une source de vérité. Dans les quatre cas (Airflow, monitoring, MLflow, et ces deux briques de gouvernance), le principe est le même : **on ne fait pas confiance à ce qu'on suppose être vrai, on vérifie contre une trace conservée**. C'est la colonne vertébrale silencieuse de toute la plateforme Anfa.

Partie 4 — Le passage au cloud réel

Le cours répète que le code est « transposable tel quel » vers un cloud public parce que chaque brique open source a un équivalent managé. Je teste cette promesse sur trois briques, avec des ordres de grandeur de coût réels (tarifs AWS, région US East, juillet 2026) plutôt que des estimations vagues.

4.1 MinIO → Amazon S3

Ce qui change. Le code applicatif ne change quasiment pas : `boto3.client("s3", endpoint_url=...)` pointe vers MinIO aujourd'hui ; il suffit de retirer `endpoint_url` (ou de le remplacer par l'URL régionale AWS) et de fournir de vraies clés IAM à la place de mes clés applicatives anfa-app-

key. Les noms de bucket (anfa-raw, anfa-processed, anfa-streaming) et la structure en préfixes restent identiques. Ce qui change réellement : la gestion des accès passe de deux paires d'identifiants (root/applicatif) à une politique IAM complète (rôles, politiques par bucket), et la durabilité/réplication devient la responsabilité d'AWS plutôt que d'un volume Docker local.

Ce qui coûte de l'argent. Le stockage lui-même (S3 Standard : **0,023 \$/Go/mois** pour les 50 premiers To), les requêtes (PUT/GET facturées à l'unité), et surtout le **transfert sortant** — gratuit en entrée, payant en sortie au-delà d'un palier gratuit mensuel. Pour un data lake Anfa de taille TP (quelques Go de CSV, Parquet et artefacts MLflow), le stockage seul resterait de l'ordre de **quelques dollars par mois** ; le poste qui grossirait vraiment en production serait les requêtes et la sortie de données vers des tableaux de bord ou une appli mobile externes.

Vendor lock-in réintroduit. L'API S3 est un standard de facto, donc la portabilité du *code* reste bonne — mais la facturation (transfert sortant payant, tarification par requête) crée un coût de sortie réel qui n'existe pas avec MinIO auto-hébergé. Migrer S3 vers un autre fournisseur plus tard impliquerait de retélécharger l'intégralité des données (payant) puis de les réuploader ailleurs.

4.2 Kind → EKS / GKE

Ce qui change. Les manifestes YAML (Deployment, Service, PersistentVolumeClaim) ne changent pas de syntaxe — c'est tout l'intérêt de Kubernetes comme standard. Ce qui change : le StorageClass par défaut de Kind (provisioner local) doit être remplacé par une classe de stockage cloud (EBS sur AWS, Persistent Disk sur GCP) ; le Service de type NodePort que j'utilise pour contourner l'absence d'exposition directe de Kind deviendrait un LoadBalancer

réel, avec une vraie IP publique facturée séparément ; et la gestion multi-nœuds, purement simulée par des conteneurs Docker sous Kind, deviendrait une vraie flotte de machines virtuelles avec autoscaling.

Ce qui coûte de l'argent. Le control plane EKS coûte **0,10 \$/heure** (~72 \$/mois), fixe, quel que soit le nombre de nœuds — un coût qui n'existe pas avec Kind (gratuit, local). S'y ajoute le coût des nœuds eux-mêmes : un t3.medium (2 vCPU / 4 Go, comparable aux ressources allouées à mon worker MinIO en séance 3) coûte environ **0,042 \$/heure**, soit ~30 \$/mois par nœud tournant en continu. Pour une topologie minimale à 2 nœuds équivalente à mon cluster de TP, le total tourne autour de **130 \$/mois**, avant même de compter le stockage EBS et le LoadBalancer.

Vendor lock-in réintroduit. Kubernetes lui-même reste portable — je pourrais recréer les mêmes manifestes sur GKE ou AKS sans les réécrire. Le verrouillage vient d'ailleurs : des ressources spécifiques au cloud (annotations IAM pour les rôles de service, classes de stockage propriétaires, intégrations de load balancer natives) s'infiltrant presque toujours dans les manifestes réels au fil du temps, même si le cœur K8s reste standard.

4.3 Spark standalone → EMR / Dataproc

Ce qui change. Le code PySpark (heures_de_pointe.py, analyse_referentiel_cluster.py) ne change pas : c'est la même API DataFrame. Ce qui change, c'est la soumission du job (spark-submit --master spark://spark-master:7077 devient une soumission via l'API/CLI EMR ou

Dataproc) et la configuration du connecteur S3A, qui pointerait vers le vrai S3 plutôt que vers mon MinIO local avec ses identifiants anfa-app-key.

Ce qui coûte de l'argent. EMR facture une **surcharge de 15 à 25 % au-dessus du prix EC2 nu** : un m5.xlarge (4 vCPU / 16 Go) coûte 0,192 \$/h sur EC2 seul, mais **0,240 \$/h avec la surcharge EMR**. Pour un cluster équivalent au mien (1 master + 2 workers), soit 3 instances, tournant en continu : $0,240 \times 3 = 0,72$ \$/h, environ **518 \$/mois** — un montant qui n'a de sens que si le cluster tourne réellement 24h/24. Or mon cluster Spark ne tourne que le temps du batch nocturne orchestré par Airflow. C'est précisément l'argument commercial d'EMR/Dataproc : facturer à la seconde et détruire le cluster entre deux exécutions ferait tomber ce coût à quelques dollars par mois pour un usage TP-scale — mais seulement si l'équipe automatise vraiment la création/destruction du cluster (ce que mon docker-compose.yml local, qui reste allumé en permanence pendant que je travaille, ne m'a jamais forcé à faire).

Vendor lock-in réintroduit. Le calcul Spark reste portable (même code, même API), mais EMR et Dataproc ajoutent des couches propriétaires (bootstrap actions AWS, images optimisées Dataproc, intégrations natives avec leurs propres outils de planification) qui, une fois adoptées, rendent la migration vers un autre cloud plus coûteuse que la seule réécriture du job Spark lui-même.

4.4 Synthèse

Sur les trois briques, le même schéma se répète : **le code survit à la migration, la facture et les détails opérationnels non**. Le vrai travail de transposition n'est pas de réécrire des scripts, c'est de redessiner qui paie quoi, quand couper les ressources, et quelles couches propriétaires on accepte d'introduire pour gagner en confort opérationnel.

Partie 5 — Souveraineté et conformité

5.1 Le scénario

La future application mobile Anfa collecterait, pour chaque passager : sa **position GPS** (pour proposer le trajet le plus proche), son **historique de paiements mobile money** (pour la gestion de l'abonnement), et son **numéro de téléphone** (identifiant de compte). Ce sont exactement les données que j'ai analysées dans ma fiche de conformité de la séance 10 (seance-10/FICHE_CONFORMITE.md).

5.2 Où héberger ces données ?

Je défends l'**hébergement on-premise à Lomé**, avec la même architecture open source que celle utilisée tout au long de ce cours (MinIO, PostgreSQL) plutôt qu'un déplacement vers un cloud public, américain ou européen.

L'argument de conformité. Anfa est une entreprise togolaise ; le texte directement applicable n'est pas le RGPD (qui ne s'applique pas hors UE) mais la **Loi n°2019-014 du 29 octobre 2019** relative à la protection des données à caractère personnel — le texte togolais analogue au RGPD, qui définit les obligations d'un responsable de traitement et crée une autorité de contrôle nationale. Les données de paiement mobile money relèvent en plus de la **Loi n°2017-007 du 22**

juin 2017, modifiée par la Loi n°2023-012 du 19 juillet 2023, relative aux transactions

électroniques. Une même donnée (le paiement mobile money d'un passager) relève donc de deux textes à la fois, et une plateforme conforme doit satisfaire les deux simultanément — ce n'est pas un détail, c'est une contrainte structurante sur la conception du pipeline.

L'argument de souveraineté. Le choix « 100 % open source, déployé localement, sans cloud commercial payant » que ce cours impose depuis la séance 1 n'est pas qu'une contrainte pédagogique et budgétaire : c'est un choix de souveraineté des données. Héberger chez un fournisseur cloud américain exposerait potentiellement les données d'Anfa au **Patriot Act** (États-Unis, 2001), quel que soit le lieu physique du datacenter. C'est un argument d'**extraterritorialité**, pas seulement de coût. Le raisonnement complet :

- Hébergement cloud américain → soumis potentiellement au Patriot Act → risque d'accès par une juridiction étrangère → conformité avec la Loi 2019-014 rendue plus complexe à démontrer.
- Hébergement local (ce que je fais depuis la séance 1) → données sous juridiction togolaise → conformité directement démontrable auprès de l'autorité nationale de contrôle.

Le compromis assumé. L'hébergement on-premise coûte plus cher en administration système (pas d'autoscaling managé, pas de SLA) et prive Anfa des économies d'échelle d'un cloud public analysées en Partie 4. Je le défends quand même parce que la donnée en jeu — géolocalisation continue et historique financier de passagers — a un niveau de sensibilité qui justifie ce surcoût pour garder un contrôle direct et démontrable.

5.3 Ce qui manque encore

Ma fiche de conformité de la séance 10 m'a forcé à un constat honnête : la plateforme Anfa **n'a pas de mécanisme réel pour retrouver et supprimer toutes les données d'un passager donné** à travers la plateforme. Le droit à l'oubli prévu par la Loi 2019-014 resterait aujourd'hui difficile à honorer en pratique. C'est exactement le principe de *privacy by design* vu en séance 10 : cette capacité doit se penser dès la conception d'un pipeline, pas s'ajouter après coup une fois la plateforme déjà construite — un axe de travail réel pour la suite du projet Anfa, pas un point théorique.

Conclusion

Dix séances, dix briques, une seule plateforme. Ce qui me frappe en écrivant ce rapport, ce n'est pas la liste des outils que j'ai appris à faire tourner — c'est le nombre de fois où le même principe est revenu sous une forme différente : garder une trace plutôt que présumer (Git, Terraform state, offsets Kafka, Model Registry), et vérifier qu'un système produit vraiment ce qu'on attend de lui plutôt que de se fier à l'absence d'erreur (idempotence Airflow, fraîcheur Prometheus, dérive de données MLflow). La plateforme Anfa que j'ai construite n'est pas parfaite - ce rapport documente aussi ses angles morts. Je les assume ici plutôt que de les taire, parce que savoir où sont les limites de ce qu'on a construit fait, je crois, autant partie du métier d'ingénieur que savoir le faire tourner.